

Customer Support Agent Evaluation

Week 4 — evaluation primer, architecture, code flow, results, and project README

Evaluation Primer

I will measure overall routing accuracy, per-class precision / recall / F1, and an LLM-judged reasoning-justification score on my LangGraph 5-way e-commerce support-ticket router using a golden dataset of 150 human-labeled tickets (order_status 24, refund_request 42, product_issue 24, account_help 30, other 30 — including hard boundary cases such as delivered-but-never-arrived + "refund please" and defective-item + refund-remedy), with exact-match against human labels for the category and LLM-as-judge (gpt-4o-mini, anchored 1–5 rubric) for reasoning quality. Pass bar: $\geq 95\%$ accuracy, no class below 0.90 precision and recall, mean reasoning score $\geq 4.0 / 5$, p95 latency under ~ 1.5 s per ticket, well under \$0.05 per 150-ticket run. I will trace in LangSmith: baseline prompt (v1) vs disambiguation-rule prompt (v2).

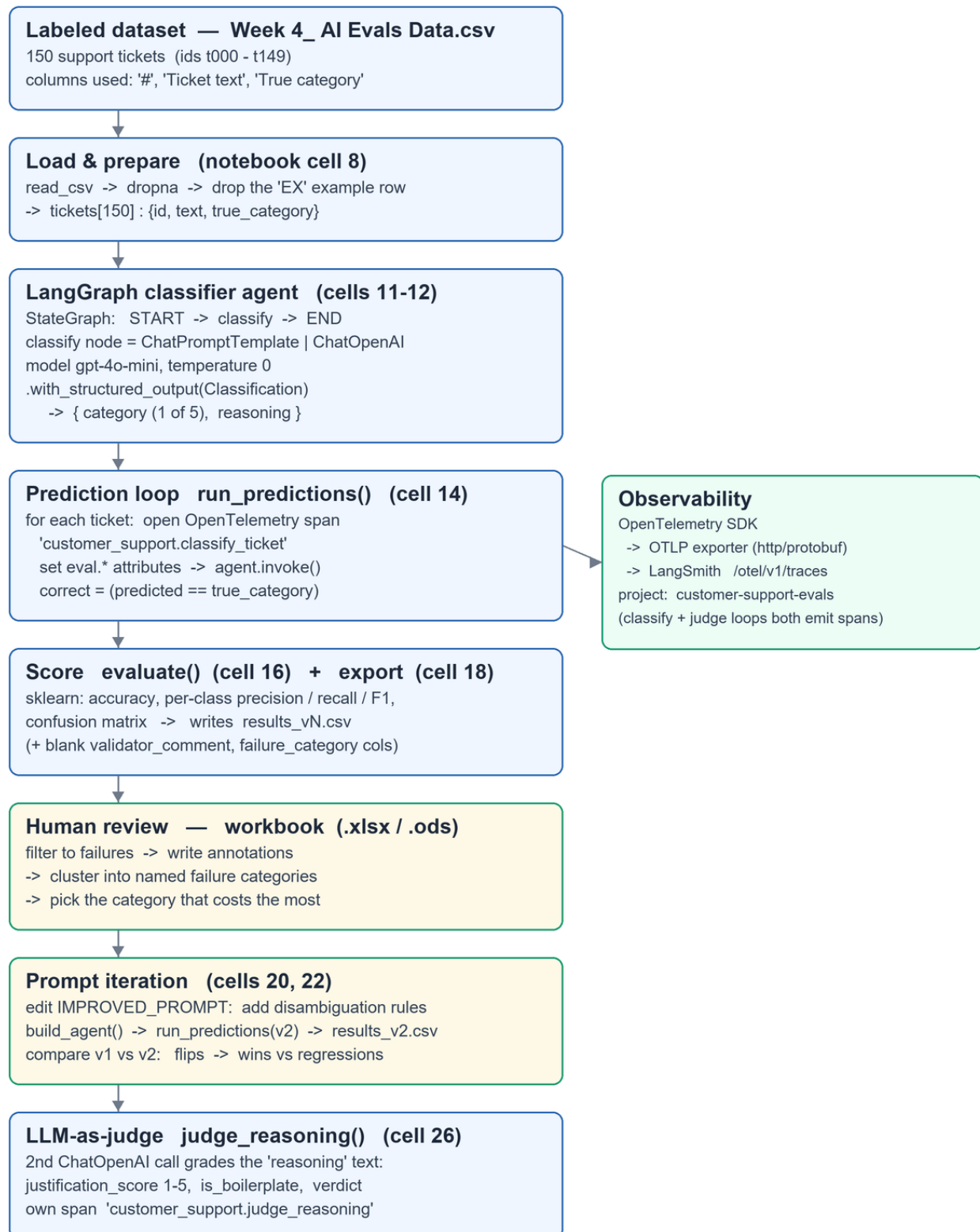
Outcome: accuracy bar met (v1 96.0%, v2 96.7%); the reasoning score and cost/latency bars were met; the "no class below 0.90" bar was *missed on one cell* — *other* precision stayed at ~ 0.86 – 0.88 because a few account_help / refund_request tickets leak into *other*.

Slot mapping

Placeholder	Value
METRIC SET	accuracy, per-class precision / recall / F1, LLM-judged reasoning score
AGENT	LangGraph 5-way e-commerce support-ticket router (gpt-4o-mini, temperature 0, structured output)
N	150 (ids t000–t149, loaded from Week 4_AI Evals Data.csv)
SCENARIO TYPES	5 categories; boundary cases: non-delivery + refund, defective-item + refund, price-adjustment, promo-code-at-checkout
JUDGE METHOD	exact-match vs human labels for the category + LLM-as-judge (anchored 1–5 rubric) for the reasoning sentence
TARGET	$\geq 95\%$ accuracy, ≥ 0.90 per-class P/R, reasoning $\geq 4.0/5$, p95 $< \sim 1.5$ s/ticket, $< \$0.05$ /run
BASELINE	vs v1 plain prompt (96.0%) vs v2 with 4 disambiguation rules + forced primary-problem step (96.7%)
POST-IMPROVEMENT	

System Architecture

Data flows top to bottom; the prediction and judge loops also stream OpenTelemetry spans sideways into LangSmith.



Components

Component	Role in the system
Dataset CSV	Week 4_ AI Evals Data.csv — 150 rows of (Ticket text, True category); the only source of ground truth.
Loader (cell 8)	pandas read_csv on 3 columns, drops empty rows and the worked "EX" row, asserts every label is one of the 5 categories, builds tickets[150].
Categories (cell 6)	CATEGORIES = order_status, refund_request, product_issue, account_help, other.
Agent (cells 11–12)	LangGraph StateGraph with one node "classify" (START -> classify -> END). The node runs ChatPromptTemplate ChatOpenAI(gpt-4o-mini, temperature 0) with .with_structured_output(Classification) so every call returns {category, reasoning}. build_agent(prompt) recompiles the graph for a new prompt.
Prediction harness (cell 14)	run_predictions() iterates tickets, opens one OpenTelemetry span per ticket (customer_support.classify_ticket), attaches eval.* attributes, calls agent.invoke(), records correctness, returns a DataFrame.
Scorer (cell 16)	evaluate() — scikit-learn accuracy_score, classification_report, confusion_matrix.
Result files (cell 18)	results_v1.csv / results_v2.csv — one row per ticket plus empty validator_comment and failure_category columns for human review.
Human workbook	Week 4_ AI Evals (...).xlsx / Tosubmit/*.ods — filter failures, annotate, cluster into failure categories, pick the costliest.
Improvement loop (cells 20, 22)	IMPROVED_PROMPT adds disambiguation rules; re-run as v2; compare merges v1/v2 on id and splits changed predictions into wins and regressions. Separate ChatOpenAI(gpt-4o-mini) call with a written rubric grades the model's own reasoning sentence (justification_score 1–5, is_boilerplate, verdict); emits customer_support.judge_reasoning spans.
LLM-as-judge (cell 26)	OpenTelemetry SDK -> OTLP exporter (http/protobuf) -> LangSmith endpoint /otel/v1/traces, project customer-support-evals. Configured in cell 4; LANGSMITH_OTEL_ENABLED=true routes spans to LangSmith.
Observability	

Code Flow

End-to-end, the executed notebook does the following (cell numbers in the first column):

Cell	What happens
2	Install the dependencies (langgraph, langsmith[otel], langchain-openai, pandas, scikit-learn, pydantic, tqdm, opentelemetry-*).
4	Configure keys and tracing. Prompt for OPENAI_API_KEY and LANGSMITH_API_KEY; clear stale OTEL_* variables; set LANGSMITH_TRACING, the project name, and the OTLP endpoint to https://api.smith.langchain.com/otel/v1/traces.
6	Define the 5 category keys.
8	Load the data. Read Week 4_ AI Evals Data.csv, drop blank rows and the "EX" row, strip whitespace, assert labels are valid → 150 tickets. Print per-class counts.
11	Build the agent. Define CLASSIFIER_PROMPT, the Classification schema (category + one-sentence reasoning) and AgentState; build_agent() compiles the LangGraph graph; smoke-test on ticket t000.
12	Get the tracer — trace.get_tracer("customer-support-evals").
14	Run the baseline. run_predictions(agent, tickets, "baseline", "v1"): per ticket open a span, set eval.example_id / run_name / prompt_version / true_category / input, call the agent, set eval.predicted_category / correct / reasoning → results_v1 DataFrame.
16	Score v1. evaluate(results_v1): accuracy, per-class precision / recall / F1, confusion matrix.
18	Export results_v1.csv with blank validator_comment and failure_category columns.
human	Review. Open the CSV / workbook, filter correct == FALSE, annotate each failure, cluster the notes into 2–4 named failure categories, pick the one most expensive in production.
20	Improve and re-run. Edit IMPROVED_PROMPT (disambiguation rules + "identify the primary problem first"); build_agent(IMPROVED_PROMPT); run_predictions(..., "improved", "v2") → results_v2 + results_v2.csv; evaluate(results_v2).

Cell	What happens
22	Compare. Merge v1 and v2 on id, isolate tickets whose prediction changed, split into wins (wrong → right) and regressions (right → wrong).
26	LLM-as-judge. judge_reasoning(results_v1): for each row a second LLM call grades the reasoning sentence on the 1–5 rubric; cross-tabulate verdict against label correctness. Each judgement is its own span.

Results (150-ticket run)

Numbers below are from the current on-disk results_v1.csv / results_v2.csv, which reflect all 150 tickets in the CSV.

Run	Accuracy	Correct
Baseline prompt (v1)	96.0%	144 / 150
Improved prompt (v2)	96.7%	145 / 150
Change	+0.7 pp	+1 net (2 wins, 1 regression)

Per-class precision / recall / F1

Category	N	P v1	R v1	F1 v1	P v2	R v2	F1 v2
order_status	24	0.96	1.00	0.98	1.00	0.96	0.98
refund_request	42	1.00	0.93	0.96	1.00	0.98	0.99
product_issue	24	1.00	1.00	1.00	0.96	1.00	0.98
account_help	30	1.00	0.90	0.95	1.00	0.90	0.95
other	30	0.86	1.00	0.92	0.88	1.00	0.94

What the prompt change did

The baseline's 6 errors: 3 refund_request tickets misrouted (t015 → order_status, t049 / t050 → other) and 3 account_help promo-code tickets (t114–t116 → other). The improved prompt added rules such as "never received the order → order_status even if money is mentioned", "damaged / wrong item → product_issue regardless of remedy", and "where is my refund?" → refund_request".

- **Wins (2):** t015 "delivered but I never got it — refund please" and t049 "price dropped after I bought it — any way to get the difference?" both moved to refund_request.
- **Regression (1):** t025 "carrier delivered to the wrong address" (true order_status) flipped to product_issue — the new "wrong item → product_issue" rule over-fired on the word "wrong".
- **Still wrong in both:** t050 (price-adjustment request read as other) and t114–t116 (newsletter promo code "invalid" at checkout read as other instead of account_help).

Note on dataset versions

The committed notebook's *saved* cell outputs (100 tickets, 92% → 97%) are from an earlier, smaller version of the dataset. The CSV now contains 150 tickets and the code loads all of them, so the figures in this document — and in results_v1.csv / results_v2.csv — are the current ones. Re-running the notebook top to bottom reproduces the 150-ticket numbers. The LLM-as-judge cell has not been re-run against the 150-ticket set.

Project README

Customer Support Agent Evaluation

This project evaluates an e-commerce customer support routing agent. Given a support ticket, the agent assigns one of five categories: order_status, refund_request, product_issue, account_help, other.

The repository includes an executed evaluation notebook, the human-review workbook, exported results, and a short guide to the OpenTelemetry integration.

Repository Contents

File	Description
week4_customer_support_evals.ipynb	Executed notebook: classifier, evaluation workflow, prompt revision, results, trace instrumentation.
Week 4_ AI Evals Data.csv	Labeled dataset the notebook loads — 150 tickets (Ticket text + True category).
Week 4_ AI Evals (E-Commerce Customer Support Agent).xlsx	Workbook for reviewing failures, grouping error patterns, revising prompts, and optionally aligning an LLM judge.
results_v1.csv / results_v2.csv	Baseline and improved-prompt predictions with evaluation results.
docs/opentelemetry_integration_one_pager.md	Overview of the OpenTelemetry and LangSmith tracing setup.

(Results and the evaluation workflow are covered in the sections above.)

OpenTelemetry and LangSmith

Each ticket classification is wrapped in an OpenTelemetry span recording the ticket ID, prompt version, expected category, predicted category, correctness, and model reasoning. LangSmith receives these traces so individual failures can be inspected alongside the aggregate evaluation results. See [docs/opentelemetry_integration_one_pager.md](#) for details and troubleshooting.

Run the Notebook

```
python3 -m venv .venv
source .venv/bin/activate
pip install jupyter langgraph "langsmith[otel]>=0.4.25" langchain-openai \
    langchain-core pandas scikit-learn pydantic tqdm \
    opentelemetry-sdk opentelemetry-exporter-otlp
```

```
export OPENAI_API_KEY="your-openai-key"
export LANGSMITH_API_KEY="your-langsmith-key"
export LANGSMITH_TRACING=true
export LANGSMITH_PROJECT=customer-support-evals
export LANGSMITH_ENDPOINT=https://api.smith.langchain.com
export LANGSMITH_OTEL_ENABLED=true
```

```
jupyter notebook week4_customer_support_evals.ipynb
```

Do not commit API keys, .env files, or other secrets.

References

- [Customer Support Agent Evaluation solution kit](#)
- [Evaluation spreadsheet](#)
- [Submission form](#)